

# Transaction Manual

| VersionNo | Date      | Description                                                 |
|-----------|-----------|-------------------------------------------------------------|
| v1.0.0    | 2025.9.23 | Replace the POI library with FinancialLib_x.x.x_release.aar |

## Catalog

[Overview](#)

[Preparation](#)

[Getting Started](#)

[Configuration](#)

[Start Transaction](#)

[Transaction Flow](#)

## Overview

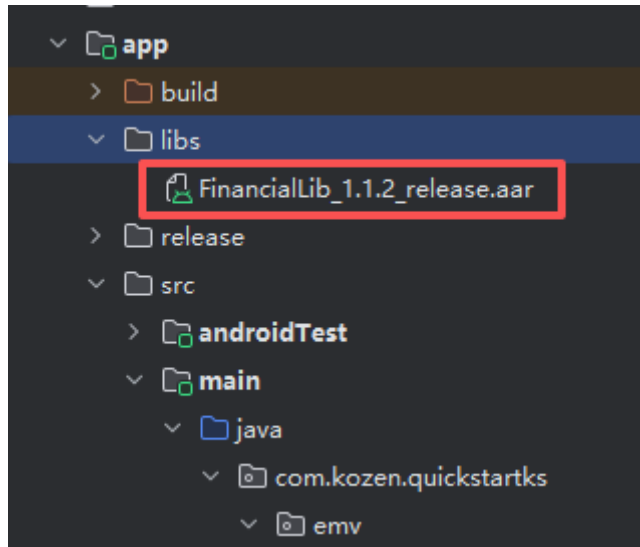
This manual is designed to help payment application developers quick-start with KOZEN OS SDK.

## Preparation

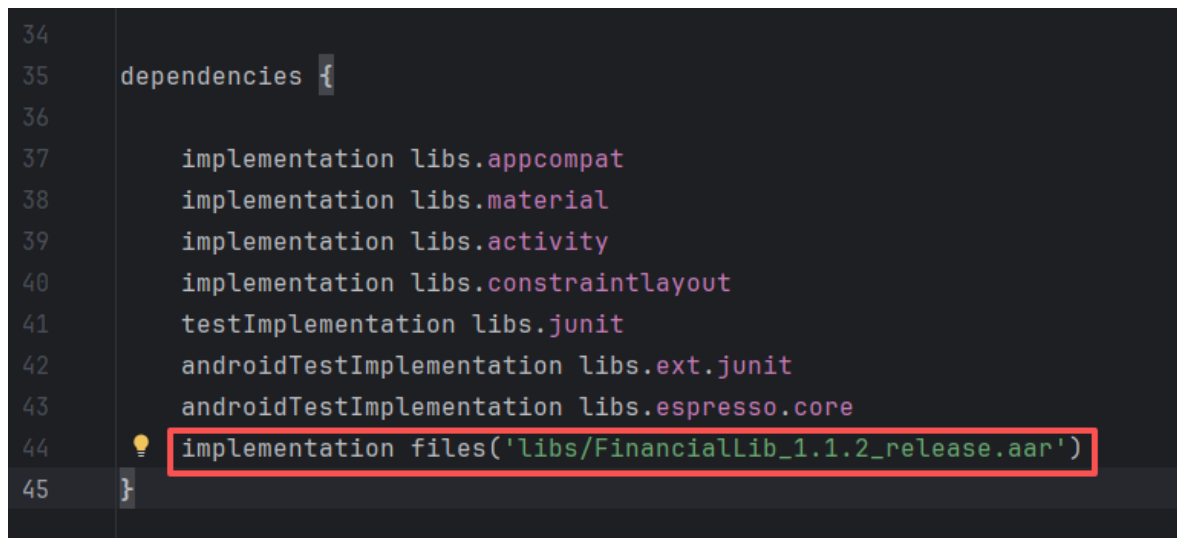
- Environment
  - ✧ JDK
  - ✧ Android SDK 7.0+
  - ✧ Android Studio
- References
  - ✧ KOZEN OS SDK
  - ✧ QuickStart Demo

# Getting Started

1. Import sdk, create libs directory in the project, copy KOZEN SDK named ' FinancialLib\_x.x.x\_release.aar ' to this directory



2. Add the dependency in the build.gradle file



# Configuration

## 1. ImportKey

### ➤ importTMK

```
FinancialEngine.INSTANCE.getSecurityManager().writeKeyMKS(K(ConstantSecurity.PED_TLK,  
srcKeyIndex,ConstantSecurity.PED_TMK,ConstantSecurity.ENCRYPTION_ALGORITHM_TDES,  
tmkIndex,keyData,kcv));|
```

### ➤ importTPK

```
FinancialEngine.INSTANCE.getSecurityManager().writeKeyMKS(K(ConstantSecurity.PED_TMK,  
tmkIndex,ConstantSecurity.PED_TPK,ConstantSecurity.ENCRYPTION_ALGORITHM_TDES,  
tpkIndex,keyData,kcv));|
```

### ➤ importIPEK

```
FinancialEngine.INSTANCE.getSecurityManager().writeKeyDukptDes(tikIndex,  
tlkIndex, keyData, ksn, kcv);
```

## 2. Import AID and CAPK

```
FinancialEngine.INSTANCE.getEmvManager().setAid(emvAid);  
FinancialEngine.INSTANCE.getEmvManager().setCapk(emvCapk);
```

## 3. Setup contactless kernel configuration

```
FinancialEngine.INSTANCE.getEmvManager().setDRL(EmvDrlConstraints.TYPE_VISA, bundle);
```

# Start Transaction

## 1. Set the transaction type and amount

```
Bundle transParam = new Bundle();  
transParam.putInt(ConstantEmv.POIEmvCoreManager.EmvTransDataConstraints.TRANS_TYPE,  
    POIEmvCoreManager.EMV_GOODS);  
transParam.putLong(EmvTransDataConstraints.TRANS_AMOUNT, amount);  
transParam.putLong(EmvTransDataConstraints.TRANS_AMOUNT_OTHER, amountOther);
```

## 2. Configure the supported transaction types

```
int transMode = 0;  
//Support Contact  
transMode |= POIEmvCoreManager.DEVICE_CONTACT;  
//Support Contactless  
transMode |= POIEmvCoreManager.DEVICE_CONTACTLESS;  
//Support Magstripe  
transMode |= POIEmvCoreManager.DEVICE_MAGSTRIPE;  
transParam.putInt(EmvTransDataConstraints.TRANS_MODE, transMode);
```

## 3. Configure the transaction timeout and other parameters

```
transParam.putInt(EmvTransDataConstraints.TRANS_TIMEOUT, 60);  
transParam.putBoolean(EmvTransDataConstraints.SPECIAL_CONTACT, true);  
transParam.putBoolean(EmvTransDataConstraints.SPECIAL_MAGSTRIPE, true);  
transParam.putBoolean(EmvTransDataConstraints.TRANS_FALLBACK, false);
```

4. Start transaction and set the transaction flow listener

```
FinancialEngine.INSTANCE.getEmvManager().startTransaction(transParam, emvListener);
```

## Transaction Flow

The transaction flow is listened by IEmvListener object, includes the following methods:

```
IEmvListener emvListener = new IEmvListener() {  
    public void onEmvProcess(int type, Bundle cardInfo) {}  
    @Override no usages  
    public void onSelectApplication(List<String> applList, boolean isFirstSelect) {}  
    @Override no usages  
    public void onConfirmCardInfo(int mode, Bundle info) {}  
    @Override no usages  
    public void onKernelType(int type) {}  
    @Override no usages  
    public void onSecondTapCard() {}  
    @Override no usages  
    public void onRequestInputPin(Bundle pinParam) {}  
    @Override no usages  
    public void onRequestOnlineProcess(Bundle onlineParam) {}  
    @Override no usages  
    public void onTransactionResult(int resultCode, Bundle transData) {}  
};
```

Methods description:

- 1.onEmvProcess: Return the card type which be detected
- 2.onSelectApplication: Do multi-AID selection
- 3.onConfirmCardInfo:Do interaction with the card at specified EMV step
- 4.onKernelType: Return the kernel type which be used
- 5.onSecondTapCard: Do second tap card
- 6.onRequestInputPin:Request pinpad, capture online-pin or offline-pin
- 7.onRequestOnlineProcess:Do online authorization
- 8.onTransactionResult: Return transaction result